Memento

Season: 3 · Behavioral · Category: GoF · Level: Intermediate · Build with Waqar

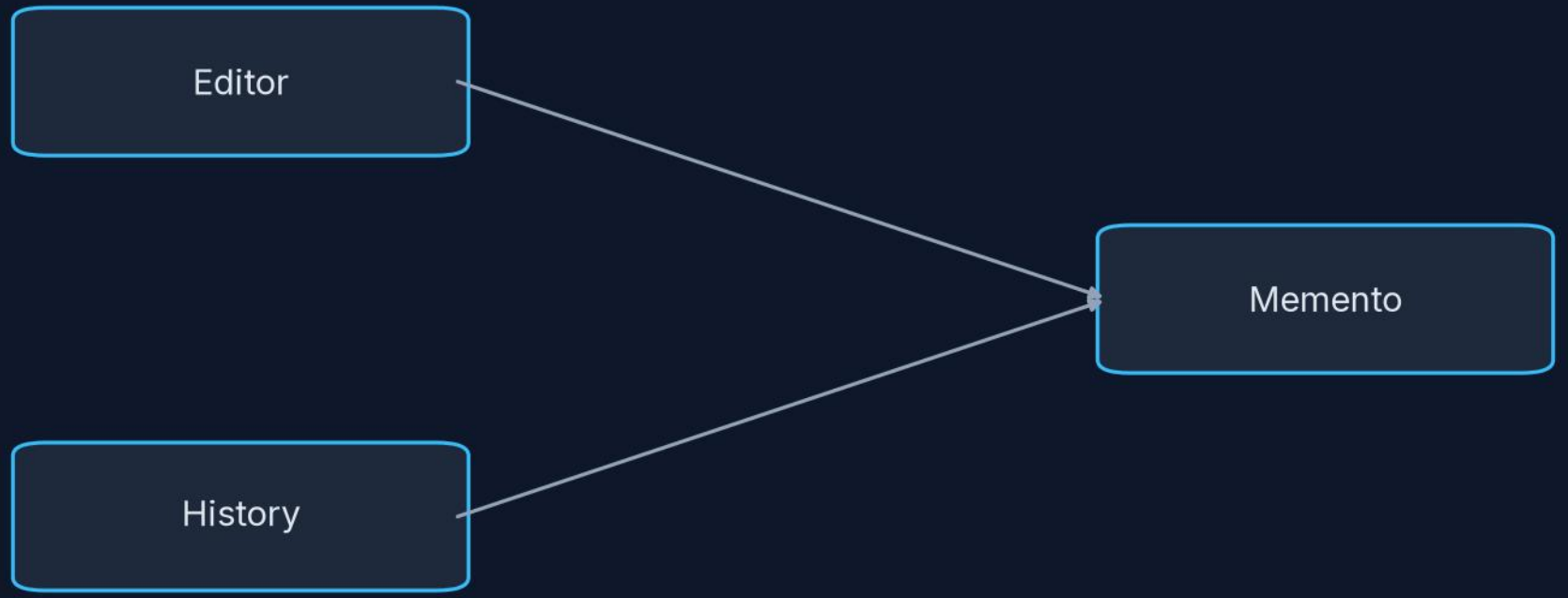
Today's Hook

- Today: Memento as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Memento structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Memento
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/gof/behavioral/memento/TextEditor.java
package com.buildwithwaqar.patterns.gof.behavioral.memento;

import java.util.ArrayDeque;
import java.util.Deque;

public final class TextEditor {
    private String text = "";

    public void type(String chunk) { text += chunk; }
    public String text() { return text; }

    public Memento save() { return new Memento(text); }
    public void restore(Memento m) { text = m.state(); }

    public record Memento(String state) {}

    /** Caretaker */
    public static final class History {
        private final Deque<Memento> stack = new ArrayDeque<>();
        public void push(Memento m) { stack.push(m); }
        public Memento pop() { return stack.pop(); }
        public boolean isEmpty() { return stack.isEmpty(); }
    }
```

Implementation (2)

```
}
```

Demo.main

```
package com.buildwithwaqar.patterns.gof.behavioral.memento;
public final class Demo {
  public static void main(String[] a) {
    System.out.println("=== Memento walkthrough ===");
    var editor = new TextEditor();
    var history = new TextEditor.History();
    editor.type("Hello");
    history.push(editor.save());
    editor.type(" World");
    history.push(editor.save());
    editor.type("!!!");
    System.out.println("now=" + editor.text());
    editor.restore(history.pop());
    System.out.println("undo1=" + editor.text());
    editor.restore(history.pop());
    System.out.println("undo2=" + editor.text());
  }
}
```

Walkthrough Output

```
=== Memento walkthrough ===
now=Hello World!!!
undo1=Hello World
undo2=Hello
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Memento
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Memento → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.